Heap Data Structures: Implementation, Analysis, and Applications

Sanya Kamra

University of the Cumberlands

MSCS 532: Algorithms and Data Structures

October 2, 2026

Abstract

This report examines binary heap data structures through two practical applications: Heapsort and a max-heap priority queue used for task scheduling. The Heapsort implementation builds a max heap in linear time and repeatedly extracts the maximum element to produce an in-place ascending ordering. The report analyzes Heapsort in the best, average, and worst cases and explains why its asymptotic running time remains O(n log n) regardless of the initial ordering of the input. A benchmark compares Heapsort with randomized Quicksort and Merge Sort on random, sorted, and reverse-sorted integer arrays. The priority queue is implemented as an array-backed binary max heap with an auxiliary task-to-index map to support efficient priority updates. The simulation demonstrates insertion, maximum extraction, key modification, and priority-based scheduling. Empirical results support the theoretical expectation that all three comparison algorithms scale near O(n log n) under the tested conditions, while the constant factors of the Python implementations differ.

Heap Data Structures: Implementation, Analysis, and Applications

A binary heap is a complete binary tree that is commonly stored in an array. Its structural property makes parent and child locations computable directly from array indices, while its ordering property allows the maximum or minimum element to be found efficiently. Max heaps are especially useful for Heapsort and for priority queues in which the task with the highest priority should be processed first. This assignment implements both applications and connects their theoretical complexity with observed execution behavior.

Heapsort Implementation

The Heapsort implementation uses a max heap. For an array index i, the left child is located at 2i + 1 and the right child at 2i + 2. The heapify operation compares a node with its children and, when necessary, swaps the node with the larger child and recursively continues down the affected subtree. This restores the max-heap property after a swap or extraction.

The build_max_heap procedure begins with the final non-leaf node and applies heapify while moving backward to the root. After the max heap has been built, the largest value is located at index 0. Heapsort swaps that maximum value with the final element of the active heap, decreases the active heap size, and applies heapify to the root. Repeating this process places the largest remaining values into their final positions from right to left.

Heapsort Time and Space Complexity

Heap Construction

Although a single heapify operation can take O(log n) time, building the heap is O(n), not O(n log n). The reason is that most nodes occur near the leaves and require little or no downward movement. Summing the work over all node heights produces a linear bound for heap construction (Cormen et al., 2022).

Best, Average, and Worst Cases

After heap construction, the algorithm performs n - 1 extraction steps. Each extraction swaps the root with the last active element and restores the heap property in O(log n) time. Therefore, the sorting phase is O(n log n). Unlike some forms of Quicksort, Heapsort does not avoid these extraction and heap-restoration steps when the data are already sorted or nearly

sorted. Consequently, the best-case, average-case, and worst-case running times are all O(n log n).

Space Complexity and Overhead

The array is rearranged in place, so the standard iterative interpretation of Heapsort requires O(1) auxiliary storage beyond the input array. The submitted heapify function is recursive, however, so its recursion stack can grow to O(log n) in the worst case. This is an implementation overhead rather than an inherent requirement of Heapsort; an iterative heapify can preserve O(1) auxiliary space. Additional overhead includes index calculations, comparisons, swaps, and function-call cost.

Empirical Comparison With Quicksort and Merge Sort

Method

The benchmark compares the custom Heapsort implementation with randomized Quicksort and Merge Sort. Input sizes of 1,000, 5,000, and 10,000 integers were tested under three distributions: random, sorted, and reverse-sorted. Every algorithm received an independent copy of the same input. Each timing was repeated three times and the average elapsed time was recorded with Python's high-resolution performance counter. Correctness was checked by comparing every result with Python's sorted output. The randomized Quicksort selects its pivot uniformly from the current subarray, reducing the systematic worst-case behavior associated with always selecting the first or last element on already ordered input.

Input Distribution	Heapsort	Randomized Quicksort	Merge Sort
Random	0.024838	0.013688	0.015121
Sorted	0.026886	0.012120	0.009315
Reverse-Sorted	0.027408	0.013418	0.010011

Table 1. Average Running Time at n = 10,000 (seconds)

The results show that all three algorithms required more time as n increased, consistent with their expected growth rates. In this implementation, Heapsort was slower than the randomized Quicksort and Merge Sort across the measured cases. That result does not contradict O(n log n) complexity because asymptotic notation suppresses constant factors and lower-order effects. The recursive heapify routine performs numerous swaps and Python-level function calls, while the particular Merge Sort and randomized Quicksort implementations have different constant costs.

Heapsort displayed relatively similar times across the three input distributions. At n = 10,000, its measured averages ranged from approximately 0.0248 to 0.0274 seconds. Randomized Quicksort also remained stable because the random pivot strategy prevented ordered input from systematically producing highly unbalanced partitions. Merge Sort performed particularly well on the sorted and reverse-sorted arrays in these runs. The measurements are machine- and run-dependent, so they should be interpreted as empirical evidence for this implementation rather than universal speed rankings.

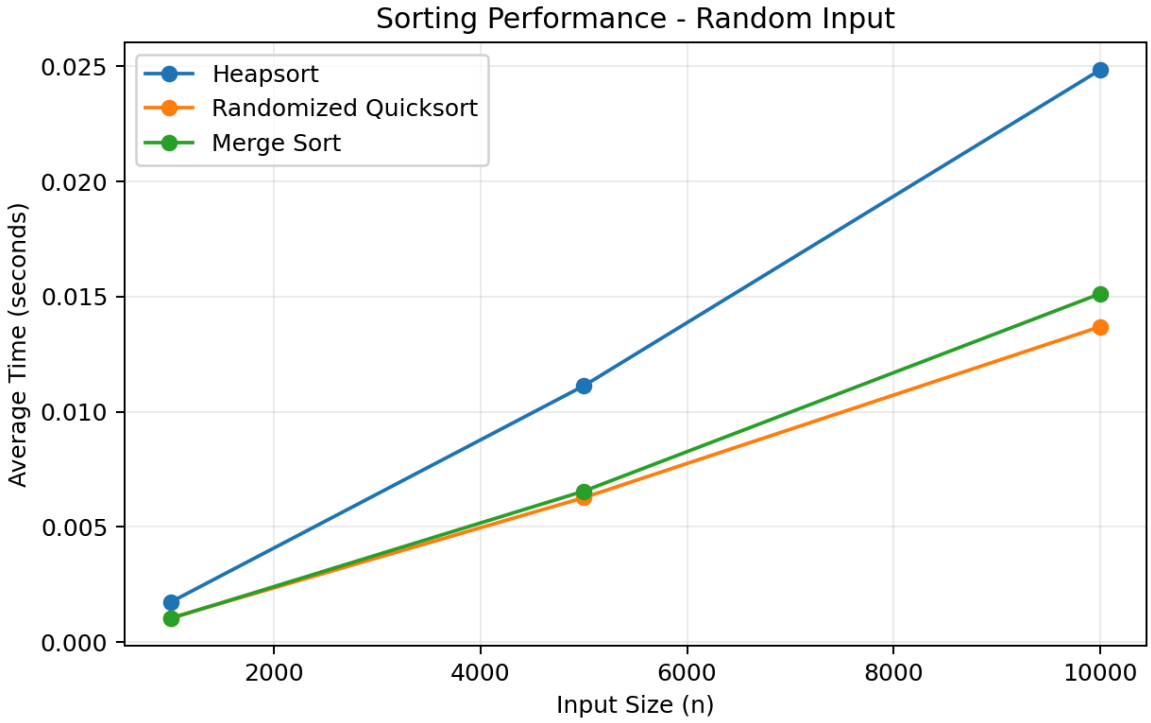


Figure 1. Runtime comparison for random input.

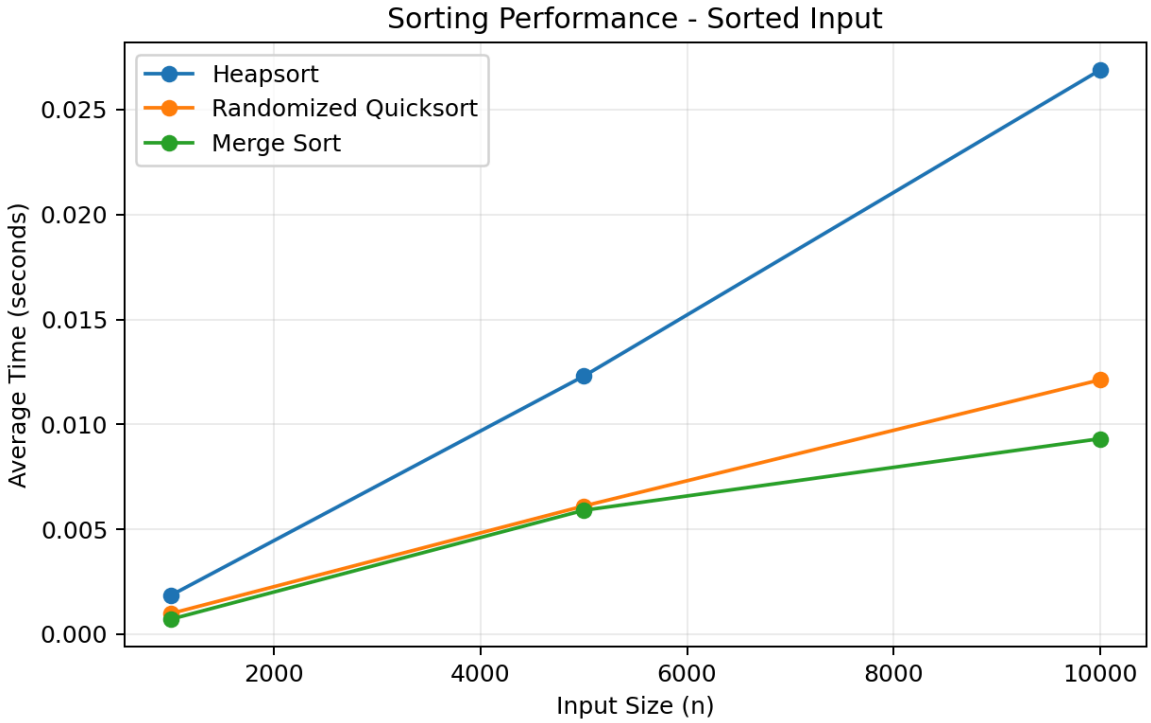


Figure 2. Runtime comparison for sorted input.

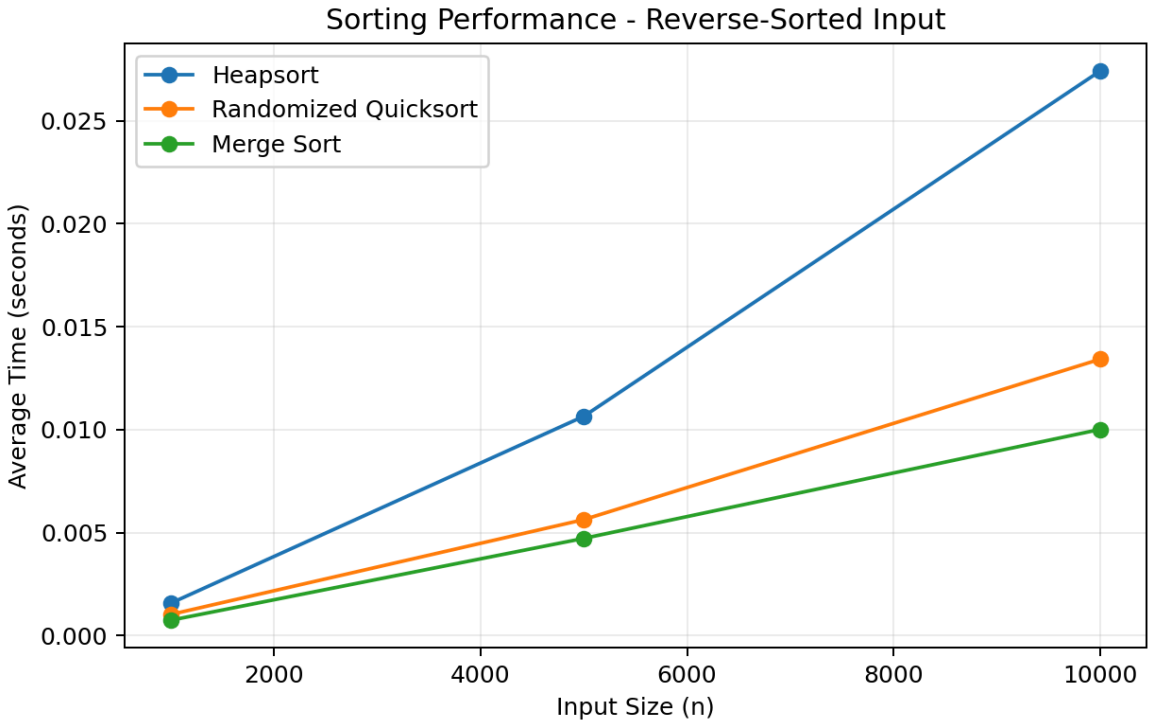


Figure 3. Runtime comparison for reverse-sorted input.

Priority Queue Design

Data Structure Selection

The priority queue uses a Python list to store a binary max heap. An array-based representation is appropriate because a binary heap is complete, so no pointer-based tree nodes are needed. Parent and child relationships are determined arithmetically, and appending a new task at the end of the list preserves the complete-tree shape before the heap property is restored. The implementation also maintains a dictionary that maps each task ID to its current heap index.

This additional structure allows increase_key and decrease_key to locate a task in O(1) expected time before repositioning it in O(log n) time.

Task Representation

Each Task object stores a task ID, numerical priority, arrival time, and optional deadline. The scheduler uses a max heap so that a larger numerical priority represents a task that should be processed sooner. Arrival time and deadline are retained as task metadata and could be incorporated into a more advanced scheduling policy, such as tie breaking or deadline-aware priority adjustment.

Priority Queue Core Operations

Table 2

Priority Queue Operation Complexity

Operation	Complexity	Reason
insert(task)	O(log n)	Append at the last position, then sift upward through at most the heap height.
extract_max()	O(log n)	Remove the root, move the final task to the root, then sift downward.
increase_key()	O(log n)	Use the index map to locate the task, change the priority, and sift upward.

Operation	Complexity	Reason
decrease_key()	O(log n)	Use the index map to locate the task, change the priority, and sift downward.
is_empty()	O(1)	Check the list length.
peek_max()	O(1)	Read the root element without removing it.

Insertion first places the task at the end of the heap and then repeatedly compares it with its parent. Extract-max removes the root and restores the heap by moving the final task to the root and sifting it downward. Increasing a key can only violate the heap property relative to ancestors, so the changed task is moved upward. Decreasing a key can only violate the property relative to descendants, so it is moved downward. Because the height of a complete binary heap is O(log n), these repositioning operations have logarithmic complexity.

Scheduler Simulation

The scheduler loads five tasks into the max-priority queue and repeatedly calls extract_max. With priorities T1 = 3, T2 = 8, T3 = 5, T4 = 10, and T5 = 2, the expected execution order is T4, T2, T3, T1, and T5. A separate priority-queue demonstration increases T1 from priority 3 to 12, causing T1 to move upward until it becomes the maximum-priority task. These examples verify that both normal extraction and priority modification preserve the max-heap invariant.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment4 % python3 scheduler.py
Priority-based execution order:
1. T4 (priority=10)
2. T2 (priority=8)
3. T3 (priority=5)
4. T1 (priority=3)
5. T5 (priority=2)
```

Figure 4. Scheduler simulation output showing priority-based execution order.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment4 % python3 priority_queue.py
Maximum before update: Task(id='T4', priority=10, arrival=3, deadline=7)
Maximum after T1 priority increase: Task(id='T1', priority=12, arrival=0, deadline=10)
Extraction order:
Task(id='T1', priority=12, arrival=0, deadline=10)
Task(id='T4', priority=10, arrival=3, deadline=7)
Task(id='T2', priority=8, arrival=1, deadline=8)
Task(id='T3', priority=5, arrival=2, deadline=12)
Task(id='T5', priority=2, arrival=4, deadline=15)
```

Figure 5. Priority queue output showing maximum-priority retrieval, priority increase, and extraction order.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment4 % python3 heapsort.py
Original: [12, 11, 13, 5, 6, 7]
Sorted:   [5, 6, 7, 11, 12, 13]
```

Figure 6. Heapsort output showing the original and sorted arrays.

```
sanyakamra@Sanyas-MacBook-Air MSCS532_Assignment4 % python3 benchmark.py
random         n= 5000 Heapsort           : 0.005328s
random         n= 5000 Randomized Quicksort: 0.003508s
random         n= 5000 Merge Sort         : 0.003688s
random         n=10000 Heapsort           : 0.009596s
random         n=10000 Randomized Quicksort: 0.006507s
random         n=10000 Merge Sort         : 0.007301s
sorted         n= 1000 Heapsort           : 0.000720s
sorted         n= 1000 Randomized Quicksort: 0.000470s
sorted         n= 1000 Merge Sort         : 0.000422s
sorted         n= 5000 Heapsort           : 0.004469s
sorted         n= 5000 Randomized Quicksort: 0.002611s
sorted         n= 5000 Merge Sort         : 0.002250s
sorted         n=10000 Heapsort           : 0.009742s
sorted         n=10000 Randomized Quicksort: 0.005549s
sorted         n=10000 Merge Sort         : 0.004833s
reverse-sorted n= 1000 Heapsort           : 0.000628s
reverse-sorted n= 1000 Randomized Quicksort: 0.000485s
reverse-sorted n= 1000 Merge Sort         : 0.000433s
reverse-sorted n= 5000 Heapsort           : 0.003915s
reverse-sorted n= 5000 Randomized Quicksort: 0.002730s
reverse-sorted n= 5000 Merge Sort         : 0.002384s
reverse-sorted n=10000 Heapsort           : 0.008860s
reverse-sorted n=10000 Randomized Quicksort: 0.005832s
reverse-sorted n=10000 Merge Sort         : 0.005057s
```

Figure 7. Benchmark output comparing Heapsort, randomized Quicksort, and Merge Sort across input distributions and sizes.

Applications

Priority queues are used whenever work must be selected according to urgency or importance rather than simple arrival order. Examples include operating-system scheduling, network packet processing, discrete-event simulation, graph algorithms such as Dijkstra's shortest-path algorithm, and job management in distributed systems. A heap is attractive because it balances fast access to the highest-priority item with logarithmic insertion and removal costs (Goodrich et al., 2014).

Conclusion

This assignment demonstrates why binary heaps are effective both for sorting and for dynamic priority management. Heapsort builds a max heap in O(n) time and completes the sorting phase in O(n log n), yielding O(n log n) time in the best, average, and worst cases. The priority queue uses the same heap structure to provide O(log n) insertion, extraction, and priority adjustment while retaining O(1) access to the maximum task and O(1) emptiness checks. The benchmark further illustrates the distinction between asymptotic complexity and measured runtime: algorithms with the same O(n log n) class can have different constant factors and practical performance. The submitted source code, benchmark results, README instructions, and scheduler simulation together satisfy the implementation, analysis, and application requirements of the assignment.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Python. Wiley.

Python Software Foundation. (2026). time — Time access and conversions. Python documentation. https://docs.python.org/3/library/time.html

Appendix A

Repository Structure

MSCS532_Assignment4/

heapsort.py

priority_queue.py

scheduler.py

benchmark.py

README.md

results/benchmark_results.csv

results/sorting_performance_random.png

results/sorting_performance_sorted.png

results/sorting_performance_reverse-sorted.png